

?# Passo a passo completo: xUnit + Moq (do zero)

Visao geral

Este guia mostra, passo a passo, como montar e entender testes automatizados com xUnit e Moq no projeto.

Meta principal:

- testar regras de negocio com seguranca,
- sem depender de banco real ativo,
- com feedback rapido ao alterar codigo.

Conceitos basicos (bem direto)

- Teste automatizado: pequeno codigo que verifica se outro codigo faz o que deveria.
- xUnit: ferramenta que executa os testes.
- Moq: ferramenta que cria "dublaes" de dependencias (mocks).
- TDD: pratica de escrever teste antes da implementacao da mudanca.

Quando usar xUnit e Moq

Use quando sua classe depende de outras camadas.

Exemplo no projeto:

- 'ClienteService' depende de 'IClientRepository'.
- Em vez de usar banco, usamos Moq para simular o repositório.

Estrutura criada

- Projeto: 'MicroERP.Tests'
- Arquivos principais:
 - 'MicroERP.Tests/Services/ClienteServiceTests.cs'
 - 'MicroERP.Tests/Services/AuthServiceTests.cs'

Passo 1 - Criar projeto de testes

Criamos um projeto separado para testes, sem misturar com a API.

Por que isso e importante:

- organiza melhor o codigo,
- evita acoplamento entre producao e teste,
- facilita rodar todos os testes com um comando.

Passo 2 - Adicionar pacotes

No '.csproj' de testes, adicionamos:

- 'xunit'
- 'xunit.runner.visualstudio'
- 'Microsoft.NET.Test.Sdk'
- 'Moq'
- 'Microsoft.EntityFrameworkCore.InMemory'

Papel de cada um:

- xUnit: cria e executa testes.
- runner/test sdk: integra execucao com CLI/IDE.
- Moq: simula interfaces.
- InMemory: cria banco temporario em memoria para cenarios que usam DbContext.

Passo 3 - Referenciar a API

Adicionamos 'ProjectReference' para 'MicroERP.Api'.

Assim os testes conseguem acessar:

- Services reais ('ClienteService', 'AuthService'),
- DTOs,
- Models,
- Excecoes.

Passo 4 - Entender o formato de um teste

Quase todo teste segue AAA:

1. Arrange (Preparar)

- cria dados de entrada,
- configura mock/contexto.

2. Act (Executar)

- chama o metodo testado.

3. Assert (Validar)

- compara resultado esperado com resultado real.

Passo 5 - Exemplo com Moq no ClienteService

No 'ClienteService', criamos:

- 'Mock<IClienteRepository>'
- 'new ClienteService(_repositoryMock.Object)'

Comandos mais usados:

- 'Setup(...)': define comportamento simulado.
- 'ReturnsAsync(...)': define retorno async.
- 'Verify(...)': confirma se metodo foi chamado (ou nao).

Exemplo mental:

- "Se buscar CPF e ja existir, entao lancar excecao".
- O mock finge que o CPF existe.
- O teste valida que a excecao aconteceu.

Passo 6 - Casos cobertos no ClienteService

CreateAsync

- CPF duplicado -> lança 'CpfAlreadyExistsException'.
- CPF novo -> cria cliente, normaliza CPF e salva.

UpdateAsync

- Cliente nao encontrado -> retorna 'null'.
- CPF novo, mas ja usado -> lança excecao.
- CPF disponivel -> atualiza e persiste.

DeleteAsync

- Cliente existe -> exclusao logica ('DeletedAt') e retorno 'true'.
- Cliente nao existe -> retorno 'false' sem salvar.

Passo 7 - Exemplo no AuthService (sem Moq)

'AuthService' usa 'AppDbContext' direto.

Nesse caso, usamos 'EF Core InMemory'.

Como funciona:

- cada teste cria um banco em memoria com nome unico ('Guid').
- prepara usuarios de teste.
- executa 'RegisterAsync' e 'LoginAsync'.
- valida resultado.

Passo 8 - Configuracao JWT para teste

Para login funcionar no teste, montamos 'IConfiguration' em memoria com:

- 'Jwt:Secret'
- 'Jwt:Issuer'
- 'Jwt:Audience'
- 'Jwt:ExpiresMinutes'

Importante:

- esse valor e de teste, nao segredo de producao.

Passo 9 - Casos cobertos no AuthService

RegisterAsync

- Email ja existe -> lança 'EmailAlreadyExistsException'.
- Email novo -> salva usuario com senha em hash.

LoginAsync

- Credenciais corretas -> retorna token.
- Senha incorreta -> retorna 'null'.

Passo 10 - Executar tudo

Comando:

```
""bash
dotnet test MicroERP.sln
""
```

Resultado atual:

- 11 testes aprovados.

Passo 11 - Como ler uma falha de teste

Quando falhar:

1. veja o nome do teste (ele descreve o comportamento esperado),
2. veja a mensagem de erro,
3. abra o teste,
4. confira o que o mock estava simulando,
5. compare com o comportamento do service.

Passo 12 - Como aplicar em novas tarefas

Sempre que for alterar regra:

1. escreva primeiro o teste do comportamento novo,
2. rode e confirme falha inicial,
3. implemente o minimo necessario,
4. rode a suite,
5. refatore com testes verdes.

Esse fluxo reduz retrabalho e aumenta confianca no codigo.

Checklist rapido

Antes de finalizar uma mudanca:

- ☐ teste novo cobre o caso principal?
- ☐ cobriu caso de erro?
- ☐ validou chamadas esperadas ('Verify') quando usa Moq?
- ☐ executou 'dotnet test'?
- ☐ suite ficou verde?

Se todos itens estiverem ok, a mudanca esta mais segura para commit.